Trainingcred
Skills for the Mordern Workforce

WORKBOOK

Get in Touch

+254-759-509615
training@trainingcred.com
www.trainingcred.com

TRANSACT-SQL AND PYTHON PROGRAMMING TRAINING — PARTICIPANT WORKBOOK

Participant Details

Name:

Organization:

Role / Title:

Email:

Training Dates:

Training Location:

WELCOME

Welcome to the Transact-SQL and Python Programming Training — a ten-day intensive programme designed exclusively for the Central Bank of Lesotho. This workbook is your active companion throughout the programme: it guides your learning during sessions, captures your analysis and decisions in structured worksheets, and travels back to your organisation as a permanent reference tool. Every page is designed to be written in, not merely read. Bring a pen, bring your questions, and bring the real data challenges you face at the Central Bank — because every exercise in this workbook is designed to be solved with your operational context in mind.

HOW TO USE THIS WORKBOOK

- Session Guides: Follow along during presentations and demonstrations — use the session notes pages to capture key concepts, fill in strategic blanks, and record instructor insights in your own words.
- Exercises and Worksheets: Complete every worksheet during the designated hands-on activity. Domain-specific column headers guide your professional thinking — resist the urge to leave rows blank.
- Reflection Journals: At the end of each day, spend the final 30 minutes writing honest, specific responses to the guided prompts. These are not summaries — they are professional commitments.
- Reference Tools: The frameworks, tools, glossary, and standards in the final section are real, currently active resources. Bookmark the URLs and return to them when you are back at your desk.
- Action Planning: The 30-60-90 Day Action Plan on Day 10 is the bridge between classroom learning and measurable workplace impact. Complete it with specific, named actions — not aspirations.

Data professionals at central banks operate in one of the most demanding technical environments in any economy: high data volumes, strict regulatory obligations, zero tolerance for errors, and constant pressure to deliver timely, accurate analytical outputs. This ten-day programme equips Central Bank of Lesotho staff with a systematic, dual-language approach to enterprise data management — combining the precision of Transact-SQL with the flexibility of Python to design, automate, secure, and optimise database solutions that meet financial sector standards. The programme moves from architectural foundations through advanced query design, automation, statistical analysis, API integration, performance monitoring, security compliance, and strategic deployment planning, culminating in a capstone presentation that simulates a real executive briefing to Central Bank leadership.

Your Learning Journey

Day	Theme	Core Focus	Signature Activity
Day 1	Foundations — Architecture, Environment, and Professional Setup	T-SQL architecture, Python basics, environment configuration, version control, and live Python-to-SQL Server connection	Integrated Environment Configuration and Live Connection Exercise
Day 2	Advanced T-SQL — Joins, CTEs, Window Functions, and Execution Plans	Complex queries, execution plan analysis, MERGE operations, error handling, and query optimisation benchmarking	Query Optimisation Data Lab with Before-and-After Performance Documentation
Day 3	Stored Procedures, Functions, Triggers, and Database Automation	Stored procedure design, user-defined functions, triggers, dynamic SQL, and secure coding practices	Automated Data Validation Stored Procedure Development
Day 4	Python Database Connectivity, Pandas, NumPy, and ETL Workflows	pyodbc, SQLAlchemy, pandas transformations, NumPy calculations, file I/O, and full ETL pipeline development	Python ETL Script: Extract, Transform, and Load to SQL Server
Day 5	Statistical Analysis, Visualization, and Midpoint Integration Simulation	SciPy statistics, Matplotlib/Seaborn visualisation, Scikit-learn fundamentals, time series, and Phase 1 Simulation	Phase 1 Simulation: Integrated T-SQL + Python Analytical Report
Day 6	Automated Reporting, Workflow Integration, and Phase 2 Simulation	Task scheduling, email automation, Jinja2 templating, configuration management, and Phase 2 Simulation	Phase 2 Simulation: Live Automated Monthly Reporting System
Day 7	API Integration, External Data Sources, and Data Quality Assurance	REST API consumption, JSON/XML parsing, web scraping, data validation, and integration testing	API Integration Exercise: Validated External Data Load to SQL Server
Day 8	Performance Monitoring, System Optimization, and Production Readiness	DMVs, Query Store, Extended Events, Python profiling, logging frameworks, capacity planning, and monitoring dashboards	Monitoring Dashboard Design Lab
Day 9	Security, Compliance, Best Practices, and Enterprise Deployment Planning	RBAC, SQL injection prevention, credential management, audit trails, code review, deployment planning, and change management	Security Assessment and Hardening Exercise
Day 10	Strategic Roadmaps, Capstone Presentations, and Programme Close	KPI definition, ROI calculation, strategic roadmap development, and executive capstone presentations	Capstone Presentation to Simulated Executive Panel

DAY 1: FOUNDATIONS — ARCHITECTURE, ENVIRONMENT, AND PROFESSIONAL SETUP

Modules: Module 1: Database Programming Fundamentals and Environment Setup

Day 1 establishes the technical and professional foundation upon which all subsequent programming work will rest. Participants configure their integrated development environments, connect Python to SQL Server, and internalize the architectural principles that govern efficient database programming. This day orients the Central Bank of Lesotho cohort to the standards of enterprise-grade database development.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30		Registration and Welcome	Administrative
09:30 - 10:30	**Module 1: Database Programming Fundamentals and Environment Setup**	T-SQL architecture, query execution principles, and optimization fundamentals in enterprise database environments	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:45	**Module 1: Database Programming Fundamentals and Environment Setup**	Python programming basics, data types, control structures, and best practices for database integration projects	Interactive Lecture
11:45 - 13:00	**Module 1: Database Programming Fundamentals and Environment Setup**	Development environment setup including SQL Server Management Studio, Python IDEs, and connection libraries configuration	Hands-on Lab
13:00 - 14:00			*Break*
14:00 - 15:00	**Module 1: Database Programming Fundamentals and Environment Setup**	Version control, documentation standards, and project structure for maintainable database programming solutions	Workshop
15:00 - 15:30	**Module 1: Database Programming Fundamentals and Environment Setup**	**★ Exercise: Configure integrated development environment and establish connection between Python and SQL Server database**	Hands-on Lab
15:30 - 15:45			*Break*
15:45 - 16:30	**Module 1: Database Programming Fundamentals and Environment Setup**	Facilitated debrief: reviewing connection results, troubleshooting environment issues across the cohort, and aligning on project standards	Facilitated Reflection
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 1 Learning Outcomes

1. Map the T-SQL query execution pipeline and identify how architectural decisions affect performance in enterprise database environments
2. Classify Python data types and control structures according to their appropriate use cases in database integration projects
3. Compare SQL Server Management Studio and Python IDE configurations to select the optimal toolchain for a given development task
4. Apply version control and documentation standards to organise a maintainable database programming project structure
5. Interpret connection library configuration outputs to diagnose and resolve environment setup issues
6. Execute the integrated environment configuration exercise to establish a verified live connection between Python and SQL Server

T-SQL Query Execution Pipeline

KEY CONCEPT

Every T-SQL query passes through four engine stages: Parsing → Algebrization → Optimization → Execution. The Query Optimizer selects an execution plan based on statistics and available indexes — understanding this pipeline is the first step toward writing queries that perform predictably at scale.

As the instructor walks through each execution stage, record the stage name, its primary function, and one architectural decision that influences its behaviour in an enterprise environment.

Execution Stage	Primary Function	Architectural Decision That Affects This Stage	Consequence of a Poor Decision Here

Your Notes:

Python Data Types and Control Structures for Database Work

PRACTITIONER NOTE

In database integration projects, choosing the wrong Python data type for a column value — for example, storing a monetary amount as a float instead of Decimal — can introduce silent rounding errors that corrupt financial calculations. Always match Python types to SQL Server column types explicitly.

Map each Python data type to its SQL Server equivalent and note the risk of a type mismatch in a Central Bank data context.

Python Data Type	SQL Server Equivalent	Mismatch Risk in Financial Data	Recommended Handling Practice

Your Notes:

Version Control and Project Structure Standards

> **ENTERPRISE STANDARD**
>
> A maintainable database programming project separates concerns into distinct folders: /sql (stored procedures and scripts), /python (ETL and automation scripts), /tests (validation scripts), /docs (data dictionaries and change logs), and /config (environment-specific settings, never committed with credentials). This structure supports code review, onboarding, and audit trail requirements.

Design your own project folder structure for a Central Bank database programming initiative. Name each folder, state its contents, and identify who is responsible for maintaining it.

Folder Name	Contents	Responsible Role	Version Control Rule (commit / exclude)

Your Notes:

WORKSHEET 1.1: Development Environment Readiness Assessment

Before and after the environment setup lab, complete this assessment for your own workstation. In the 'Pre-Lab Status' column record what was already installed or configured. In the 'Post-Lab Status' column record the verified outcome after the exercise. Use the 'Blocker / Resolution' column to document any issue encountered and how it was resolved — this becomes your personal troubleshooting reference.

Environment Component	Required Version / Configuration	Pre-Lab Status	Post-Lab Status	Blocker Encountered	Resolution Applied

WORKSHEET 1.2: Python-to-SQL Server Connection Configuration Log

During the connection exercise, record every configuration parameter you set and the outcome of each connection attempt. This log serves as your personal runbook for re-establishing or troubleshooting connections in your Central Bank environment after the programme.

Connection Parameter	Value Used	Library (pyodbc / SQLAlchemy)	Connection Attempt Outcome	Error Message (if any)	Corrective Action Taken

DAY 1 REFLECTION JOURNAL

> Take 15 minutes at the end of Day 1 to write honest, specific responses to the prompts below. These are not summaries — they are professional commitments. Reference the actual worksheets and exercises you completed today.

Looking at your Worksheet 1.1 Environment Readiness Assessment, which component took the longest to configure and what does that tell you about your organisation's current infrastructure readiness for database programming projects?

In the connection configuration log (Worksheet 1.2), you recorded at least one blocker or configuration decision. How would you document and communicate that decision to a colleague who needs to replicate your environment at the Central Bank?

The T-SQL execution pipeline has four stages. Which stage do you think is most frequently misunderstood by query writers in your team, and what is one concrete step you could take to address that gap when you return to work?

By the end of today's session you had a verified live connection between Python and SQL Server. What is the first real Central Bank dataset you would want to query through that connection, and why?

DAY 2: ADVANCED T-SQL — JOINS, CTEs, WINDOW FUNCTIONS, AND EXECUTION PLANS

Modules: Module 2: Advanced T-SQL Query Design and Optimization

Day 2 moves participants into the analytical power of advanced T-SQL, equipping them to write sophisticated queries that are both functionally correct and performance-conscious. By examining execution plans and applying index strategies, participants begin to think like database engineers rather than mere query writers. This analytical mindset is essential for supporting the Central Bank's data-intensive reporting requirements.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 2: Advanced T-SQL Query Design and Optimization**	Day 2 orientation: recap of Day 1 environment setup, introduction to advanced T-SQL design principles and the day's performance benchmarking approach	Facilitated Reflection
09:30 - 10:30	**Module 2: Advanced T-SQL Query Design and Optimization**	Complex JOIN operations, subqueries, CTEs, and window functions for sophisticated data analysis and reporting	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:45	**Module 2: Advanced T-SQL Query Design and Optimization**	Query execution plans, index utilization strategies, and performance tuning methodologies for large-scale databases	Demonstration
11:45 - 13:00	**Module 2: Advanced T-SQL Query Design and Optimization**	Advanced data manipulation using MERGE statements, table expressions, and bulk operations for efficient data processing	Guided Practice
13:00 - 14:00			*Break*
14:00 - 15:00	**Module 2: Advanced T-SQL Query Design and Optimization**	Error handling, transaction management, and concurrency control for robust database operations	Case Study Analysis
15:00 - 15:30	**Module 2: Advanced T-SQL Query Design and Optimization**	**★ Exercise: Optimize poorly performing queries and document measurable performance improvements with execution plan analysis**	Data Lab
15:30 - 15:45			*Break*
15:45 - 16:30	**Module 2: Advanced T-SQL Query Design and Optimization**	Peer review of query optimisation outputs: participants exchange execution plan analyses, validate improvement claims, and consolidate findings	Peer Review
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 2 Learning Outcomes

1. Apply complex JOIN operations, CTEs, and window functions to solve multi-layered data analysis and reporting problems
2. Interpret query execution plans to identify index utilisation gaps and performance bottlenecks in large-scale databases
3. Analyse MERGE statements and bulk operation patterns to evaluate their suitability for high-volume data processing scenarios

4. Compare error handling and transaction management strategies to select the most robust approach for concurrent database operations
5. Evaluate poorly performing queries against execution plan evidence to prioritise optimisation interventions
6. Execute the query optimisation exercise and document measurable before-and-after performance improvements with quantified metrics

CTEs, Window Functions, and JOIN Patterns

> **KEY DISTINCTION**
>
> A Common Table Expression (CTE) improves readability and supports recursive queries but does not inherently improve performance — the optimizer may still execute the underlying logic multiple times. A window function (OVER clause) computes aggregates across a defined partition without collapsing rows, making it ideal for running totals, rankings, and period-over-period comparisons in financial reporting.

For each T-SQL construct below, record a realistic Central Bank use case, the performance consideration to be aware of, and whether an index can directly support it.

T-SQL Construct	Central Bank Use Case	Performance Consideration	Index Support (Yes / Partial / No)	Preferred Alternative When Overused

Your Notes:

Reading Execution Plans: Key Operators

> **DIAGNOSTIC RULE**
>
> In an execution plan, a Table Scan or Clustered Index Scan on a large table is a warning sign — it means the optimizer could not use a selective index. Look for thick arrows (high row estimates), Sort operators (memory-intensive), and Hash Match joins (often indicating missing indexes or statistics that are out of date).

During the execution plan demonstration, record each operator the instructor highlights, what it signals about query behaviour, and the recommended corrective action.

Execution Plan Operator	What It Signals	Typical Root Cause	Recommended Corrective Action

Execution Plan Operator	What It Signals	Typical Root Cause	Recommended Corrective Action

Your Notes:

WORKSHEET 2.1: Query Performance Optimisation Benchmark Log

For each query you optimise in the Data Lab exercise, record the original and revised versions (or a brief description), the execution plan operators that changed, and the measured performance metrics before and after. Use SQL Server Management Studio's 'Include Actual Execution Plan' and 'Client Statistics' features to capture the numbers. This log is your evidence base for demonstrating measurable improvement — a skill directly transferable to performance reporting at the Central Bank.

Query ID	Original Performance Issue (from Execution Plan)	Optimisation Applied (index / rewrite / hint)	Logical Reads Before	Logical Reads After	Elapsed Time Before (ms)	Elapsed Time After (ms)	% Improvement

WORKSHEET 2.2: T-SQL Construct Selection Matrix — Query Design Priority Quadrant

Map each T-SQL construct or pattern you encountered today into the quadrant based on two axes: Query Complexity (how difficult the construct is to write and maintain) and Performance Impact (how significantly it affects execution speed on large datasets). Use this quadrant to guide your query design decisions when you return to the Central Bank. Write the construct name in the appropriate quadrant box.

PERFORMANCE IMPACT (Low → High) ↑

HIGH IMPACT / LOW COMPLEXITY *(Default choice — use these constructs first in any query design)*	**HIGH IMPACT / HIGH COMPLEXITY** *(Use with peer review — document rationale and test thoroughly before production)*

LOW IMPACT / LOW COMPLEXITY *(Acceptable for simple reporting queries — avoid over-engineering)*	**LOW IMPACT / HIGH COMPLEXITY** *(Avoid — refactor or replace with a simpler, higher-impact alternative)*

QUERY COMPLEXITY (Low → High) →

List your stakeholders and their quadrant placement below:

WORKSHEET 2.3: Error Handling and Transaction Management Design Checklist

During the case study analysis on error handling and concurrency, use this checklist to evaluate the robustness of a database operation design. For each criterion, mark whether the design you are reviewing meets the standard, partially meets it, or fails it, and note the specific gap or evidence.

Design Criterion	Met / Partial / Fail	Evidence or Gap Identified	Recommended Corrective Action	Priority (High / Medium / Low)

✏ DAY 2 REFLECTION JOURNAL

> Use the final 30 minutes of Day 2 to reflect specifically on the query optimisation work you completed today. Reference your Worksheet 2.1 benchmark log in your responses.

In your Worksheet 2.1 benchmark log, which query showed the largest percentage improvement and what single change produced that result? What does this tell you about where performance bottlenecks most commonly hide in your current database environment?

During the peer review session, your colleague validated (or challenged) your execution plan analysis. What did that external perspective reveal that you had missed in your own review?

Looking at your Query Design Priority Quadrant (Worksheet 2.2), which construct in the 'High Impact / High Complexity' box is most relevant to a reporting query you currently run at the Central Bank? What would it take to implement it safely?

The case study on error handling presented a concurrency scenario. Describe one real transaction in your Central Bank database environment where inadequate concurrency control could produce incorrect financial data, and what TRY/CATCH/ROLLBACK pattern would mitigate it.

DAY 3: STORED PROCEDURES, FUNCTIONS, TRIGGERS, AND DATABASE AUTOMATION

Modules: Module 3: Stored Procedures, Functions, and Database Automation

Day 3 elevates participants from query writing to database engineering by introducing reusable, automated database objects. Stored procedures, user-defined functions, and triggers form the backbone of enterprise-grade automation, and participants build these with security-conscious dynamic SQL practices. This capability directly supports the Central Bank's need for reliable, auditable automated database operations.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 3: Stored Procedures, Functions, and Database Automation**	Day 3 orientation: connecting automation concepts to the Central Bank's operational context and reviewing Day 2 optimisation benchmarks	Facilitated Reflection
09:30 - 10:30	**Module 3: Stored Procedures, Functions, and Database Automation**	Stored procedure design patterns, parameter handling, and return value strategies for flexible, reusable database operations	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:45	**Module 3: Stored Procedures, Functions, and Database Automation**	User-defined functions, table-valued functions, and scalar functions for modular data processing and business logic implementation	Guided Practice

Time	Module	Session / Activity	Method
11:45 - 13:00	**Module 3: Stored Procedures, Functions, and Database Automation**	Triggers for data validation, audit trails, and automated business rule enforcement across database operations	Hands-on Lab
13:00 - 14:00			*Break*
14:00 - 15:00	**Module 3: Stored Procedures, Functions, and Database Automation**	Dynamic SQL generation, SQL injection prevention, and secure coding practices for robust database applications	Workshop
15:00 - 15:30	**Module 3: Stored Procedures, Functions, and Database Automation**	**★ Exercise: Develop automated data validation stored procedure with comprehensive error handling and logging capabilities**	Hands-on Lab
15:30 - 15:45			*Break*
15:45 - 16:30			*Group Exercise*
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 3 Learning Outcomes

1. Apply stored procedure design patterns and parameter handling strategies to build flexible, reusable database operation modules
2. Classify user-defined functions — scalar versus table-valued — according to their appropriate role in modular data processing logic
3. Analyse trigger behaviour across insert, update, and delete events to evaluate their suitability for audit trail and rule enforcement scenarios
4. Compare dynamic SQL generation approaches against SQL injection prevention requirements to select secure implementation patterns
5. Interpret execution logs from a stored procedure to diagnose error handling gaps and implement corrective logging capabilities
6. Develop an automated data validation stored procedure with comprehensive error handling and logging that meets enterprise quality standards

Stored Procedure Design Patterns

> **DESIGN PRINCIPLE**
>
> A well-designed stored procedure does one thing, does it completely, and fails loudly. Use OUTPUT parameters for status codes, TRY/CATCH blocks for all DML operations, and a dedicated logging table to record every execution — including successful ones. In a regulated environment like a central bank, silent failures are more dangerous than loud ones.

For each stored procedure design pattern introduced in the lecture, record its name, its primary use case in a Central Bank context, and the parameter handling approach it requires.

Design Pattern	Central Bank Use Case	Input Parameter Strategy	Output / Return Value Strategy	Error Handling Requirement

Design Pattern	Central Bank Use Case	Input Parameter Strategy	Output / Return Value Strategy	Error Handling Requirement

Your Notes:

Trigger Design: Audit Trails and Business Rule Enforcement

> **AUDIT REQUIREMENT**
>
> AFTER triggers on DML operations (INSERT, UPDATE, DELETE) are the standard mechanism for building audit trails in SQL Server. The INSERTED and DELETED pseudo-tables capture the before and after state of every affected row. For Central Bank compliance, every trigger-based audit log must capture: the affected table, the operation type, the old value, the new value, the database user, and the timestamp.

Design the audit log table structure your trigger will write to. For each column, specify the data type and the source (INSERTED table, DELETED table, system function, or application context).

Audit Log Column	SQL Server Data Type	Source (INSERTED / DELETED / System Function)	Business Justification for Capturing This Field

Your Notes:

WORKSHEET 3.1: Stored Procedure and Function Inventory Template

During the guided practice and hands-on lab sessions, document each database object you design or review. This inventory becomes a reusable reference for your Central Bank database object catalogue — a deliverable that supports code review, onboarding, and audit trail requirements.

Object Name	Object Type (SP / Scalar UDF / TVF / Trigger)	Business Purpose	Input Parameters and Data Types	Output / Return Value	Error Handling Mechanism	Logging Table Written To

Object Name	Object Type (SP / Scalar UDF / TVF / Trigger)	Business Purpose	Input Parameters and Data Types	Output / Return Value	Error Handling Mechanism	Logging Table Written To

WORKSHEET 3.2: Dynamic SQL Security Review Checklist

For each dynamic SQL statement you write or review during the workshop, complete one row of this checklist. The goal is to confirm that every dynamic SQL pattern in your codebase meets the injection prevention and secure coding standards discussed in today's session. Carry this checklist back to the Central Bank as a code review tool.

Procedure / Script Name	Dynamic SQL Pattern Used	Parameterised (sp_executesql) ? Y/N	User Input Sanitised? Y/N	Least-Privilege Role Applied? Y/N	Injection Risk Rating (High/Med/Low)	Remediation Required

✏ DAY 3 REFLECTION JOURNAL

> Reflect specifically on the stored procedure you built in today's hands-on exercise and the group walkthrough that followed. Be honest about gaps in your current approach.

In the group walkthrough, another team challenged your SQL injection prevention approach. Was their challenge valid? What specific change would you make to your stored procedure's dynamic SQL section as a result?

Looking at your Stored Procedure and Function Inventory (Worksheet 3.1), how many of the database objects currently in production at the Central Bank do you believe have comprehensive error handling and logging? What is your estimate based on, and what would it take to verify it?

Triggers can enforce business rules automatically, but they can also create hidden performance overhead and debugging complexity. Describe one specific business rule at the Central Bank that you would implement as a trigger and one that you would deliberately keep in application logic instead — and explain why.

By the end of today you had built an automated data validation stored procedure. What is the first real validation rule from a Central Bank data process that you would encode into a production version of that procedure when you return?

DAY 4: PYTHON DATABASE CONNECTIVITY, PANDAS, NUMPY, AND ETL WORKFLOWS

Modules: Module 4: Python Data Manipulation and Database Connectivity

Day 4 bridges the T-SQL world with Python's data manipulation ecosystem, giving participants the skills to extract, transform, and load data programmatically. Working with pyodbc, SQLAlchemy, pandas, and NumPy, participants build the ETL pipelines that automate repetitive data workflows. For the Central Bank of Lesotho, this capability translates directly into faster, more reliable data preparation for analytical and reporting processes.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 4: Python Data Manipulation and Database Connectivity**	Day 4 orientation: framing the Python-to-SQL Server connectivity challenge and mapping today's ETL work to real Central Bank data pipeline scenarios	Facilitated Reflection
09:30 - 10:30	**Module 4: Python Data Manipulation and Database Connectivity**	Python database connectivity using pyodbc, SQLAlchemy, and pandas for efficient data extraction and manipulation	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:45	**Module 4: Python Data Manipulation and Database Connectivity**	Pandas DataFrame operations, data cleaning techniques, and transformation workflows for analytical data preparation	Hands-on Lab
11:45 - 13:00	**Module 4: Python Data Manipulation and Database Connectivity**	NumPy arrays, mathematical operations, and statistical calculations for quantitative analysis and data validation	Guided Practice
13:00 - 14:00			*Break*
14:00 - 15:00	**Module 4: Python Data Manipulation and Database Connectivity**	File I/O operations, CSV/Excel processing, and data format conversions for integrated data workflows	Data Lab

Time	Module	Session / Activity	Method
15:00 - 15:30	**Module 4: Python Data Manipulation and Database Connectivity**	**★ Exercise: Create Python script that extracts data from SQL Server, performs transformations, and loads results back to database**	Hands-on Lab
15:30 - 15:45			*Break*
15:45 - 16:30	**Module 4: Python Data Manipulation and Database Connectivity**	Collaborative troubleshooting: pairs diagnose each other's ETL scripts, identify transformation errors, and validate final database load results	Peer Review
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 4 Learning Outcomes

1. Apply Python database connectivity libraries — pyodbc, SQLAlchemy, and pandas — to extract and load data from a live SQL Server instance
2. Execute pandas DataFrame transformation workflows to clean and reshape raw financial datasets into analysis-ready structures
3. Analyse NumPy array operations and statistical calculations to validate the mathematical integrity of transformed data
4. Interpret file I/O outputs across CSV and Excel formats to diagnose data format conversion errors in integrated workflows
5. Compare connection management strategies in pyodbc versus SQLAlchemy to evaluate performance and reliability trade-offs
6. Develop a Python ETL script that extracts SQL Server data, performs documented transformations, and loads verified results back to the database

pyodbc vs SQLAlchemy: Connection Strategy Comparison

> **CONNECTIVITY DECISION**
>
> pyodbc provides direct, low-level ODBC access — fast for simple queries and bulk loads, but requires manual connection and cursor management. SQLAlchemy adds an abstraction layer (ORM or Core) that supports connection pooling, dialect portability, and integration with pandas read_sql() and to_sql() — making it the preferred choice for ETL pipelines that need reliability and maintainability in production.

Compare pyodbc and SQLAlchemy across the criteria below. Use today's lecture and lab experience to populate each cell with a specific, practical observation.

Comparison Criterion	pyodbc Behaviour	SQLAlchemy Behaviour	Recommended Choice for Central Bank ETL

Your Notes:

Pandas Transformation Workflow Patterns

> **DATA QUALITY RULE**
>
> Before any DataFrame is loaded back to SQL Server, apply a three-step validation gate: (1) check for null values in non-nullable columns using df.isnull().sum(), (2) verify data types match the target table schema using df.dtypes, and (3) confirm row counts match the expected extraction count. Document all three checks in your ETL script's log output.

Record each pandas transformation operation you apply in the hands-on lab, the business reason for the transformation, and the validation check that confirms it was applied correctly.

Transformation Operation	pandas Method Used	Business Reason (Central Bank Context)	Validation Check Applied	Expected Output

Your Notes:

WORKSHEET 4.1: ETL Pipeline Design and Execution Log

Document your Python ETL script design and execution results in this log. Complete the design columns before writing code, then fill in the execution columns after running the script. This log is your primary evidence artefact for the Day 4 exercise and will be referenced in the Day 10 Capstone.

ETL Stage	SQL Server Object / Python Method Used	Transformation Logic Applied	Row Count In	Row Count Out	Validation Check Result	Error Encountered and Resolution

WORKSHEET 4.2: ETL Pipeline Decision Log — Crisis and Error Escalation Record

During the collaborative troubleshooting session, record every error or unexpected behaviour encountered in your own or your partner's ETL script. For each issue, document the error message, the root cause diagnosis, the resolution applied, and the

preventive measure you would add to the script to avoid recurrence. This log trains the diagnostic thinking required for maintaining production ETL pipelines at the Central Bank.

Error / Unexpected Behaviour	Time Identified	Root Cause Diagnosis	Resolution Applied	Preventive Measure Added to Script	Ethical / Compliance Consideration (e.g. data exposure risk)

WORKSHEET 4.3: Ethical Dilemma — Data Transformation Integrity in Financial Reporting

Work through the following scenario using the structured ethical analysis framework. The scenario: During an ETL run, your Python script silently drops 847 rows because they fail a data type conversion. The resulting dataset is loaded to the reporting database and used to generate a regulatory submission report. The row count discrepancy is within the 1% tolerance your team informally accepts. Analyse this dilemma using the framework below.

Dilemma Description — State the specific ethical tension in your own words

Stakeholders Affected and Their Interests — Who is affected and what do they stand to lose or gain?

Option A: Accept the submission as-is and document the tolerance applied — Consequences

Option B: Halt the submission, investigate the 847 dropped rows, and resubmit — Consequences

Guiding Principles — Which data governance, regulatory, or professional standards apply to this decision?

Team Decision and Rationale — What would your team do and why?

✏ DAY 4 REFLECTION JOURNAL

> Day 4 introduced the full ETL pipeline — from extraction through transformation to load. Reflect on the specific decisions you made in your script and what they reveal about your current data engineering practices.

In your ETL Pipeline Design and Execution Log (Worksheet 4.1), which transformation stage had the largest row count discrepancy between input and output? What caused it, and how would you handle that discrepancy in a production regulatory reporting pipeline at the Central Bank?

The ethical dilemma in Worksheet 4.3 involved silently dropped rows in a regulatory submission. How does your team currently handle data quality exceptions in ETL processes, and does that practice meet the standard you would now apply after today's discussion?

During the collaborative troubleshooting session, your partner identified an error in your script that you had not noticed. What type of error was it — logic, syntax, data type, or connectivity — and what does that reveal about the testing gaps in your current scripting practice?

SQLAlchemy's connection pooling was introduced as a reliability feature for production ETL. Identify one specific ETL process currently running at the Central Bank that would benefit from connection pooling, and describe what failure mode it would prevent.

DAY 5: STATISTICAL ANALYSIS, VISUALIZATION, AND MIDPOINT INTEGRATION SIMULATION

Modules: Module 5: Statistical Analysis and Data Visualization with Python

Day 5 marks the pivotal midpoint of the programme, where analytical capability meets visual communication. Participants apply SciPy, Matplotlib, Seaborn, Scikit-learn, and time series techniques to transform raw data into compelling,

insight-rich reports. The Phase 1 Simulation challenges teams to integrate everything learned so far — T-SQL extraction, Python transformation, and statistical visualisation — in a realistic Central Bank analytical scenario.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 5: Statistical Analysis and Data Visualization with Python**	Day 5 orientation: midpoint programme review, framing the Phase 1 Simulation challenge, and introducing today's analytical reporting objectives	Facilitated Reflection
09:30 - 10:30	**Module 5: Statistical Analysis and Data Visualization with Python**	Statistical analysis using SciPy, descriptive statistics, hypothesis testing, and correlation analysis for data insights	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:45	**Module 5: Statistical Analysis and Data Visualization with Python**	Data visualization with Matplotlib and Seaborn for creating professional charts, graphs, and analytical dashboards	Hands-on Lab
11:45 - 13:00	**Module 5: Statistical Analysis and Data Visualization with Python**	Machine learning fundamentals using Scikit-learn for predictive modeling and pattern recognition in business data	Guided Practice
13:00 - 14:00			*Break*
14:00 - 14:45	**Module 5: Statistical Analysis and Data Visualization with Python**	Time series analysis techniques for forecasting, trend analysis, and seasonal pattern identification	Data Lab
14:45 - 15:30	**Module 5: Statistical Analysis and Data Visualization with Python**	**★ Exercise: Develop comprehensive analytical report combining T-SQL data extraction with Python statistical analysis and visualization**	Simulation & Debrief
15:30 - 15:45			*Break*
15:45 - 16:30			*Group Exercise*
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 5 Learning Outcomes

1. Apply SciPy statistical functions to execute descriptive statistics, hypothesis testing, and correlation analysis on financial datasets
2. Interpret Matplotlib and Seaborn chart outputs to evaluate whether visualisations accurately represent underlying data distributions and trends
3. Analyse Scikit-learn model outputs to classify patterns in business data and assess the predictive reliability of the results
4. Apply time series decomposition techniques to identify trend, seasonal, and residual components in financial time data
5. Develop a comprehensive analytical report that integrates T-SQL data extraction with Python statistical analysis and professional visualisation
6. Evaluate the end-to-end integrated pipeline produced in the Phase 1 Simulation against defined accuracy and performance benchmarks

Statistical Analysis with SciPy: Choosing the Right Test

STATISTICAL DECISION RULE

Choosing the wrong statistical test produces conclusions that are technically computed but analytically meaningless. For financial data: use descriptive statistics (mean, median, std) for distribution characterisation; use Pearson correlation for linear relationships between continuous variables; use the t-test for comparing two group means; use chi-square for categorical independence. Always state your null hypothesis before running any test.

For each statistical technique introduced today, record the SciPy function used, the type of question it answers, and a realistic Central Bank analytical question it could address.

Statistical Technique	SciPy Function	Question Type It Answers	Central Bank Analytical Question	Key Output to Interpret

Your Notes:

Phase 1 Simulation: Planning Template

SIMULATION BRIEF

The Phase 1 Simulation requires your team to deliver a complete analytical report within the session window. The report must include: a T-SQL extraction query with documented execution plan, at least two pandas transformation steps with validation checks, one statistical test with interpretation, and two visualisations (one distribution chart, one trend or comparison chart). All outputs must be reproducible from a single Python script.

Before starting the simulation, complete this planning template as a team. Assign each component to a team member and set a time target for completion.

Report Component	T-SQL / Python Method	Assigned Team Member	Time Target (minutes)	Acceptance Criterion

Your Notes:

WORKSHEET 5.1: Analytical Report Quality Assessment Rubric

During the Phase 1 Simulation Debrief, use this rubric to assess your own team's report and one other team's report. Score each criterion from 1 (does not meet standard) to 4 (exceeds standard). Record specific evidence for each score. This rubric will be reused in the Day 10 Capstone peer assessment.

Assessment Criterion	Self-Score (1-4)	Self-Score Evidence	Peer Team Score (1-4)	Peer Team Evidence	Improvement Action

WORKSHEET 5.2: Visualisation Design Decision Log

For each chart or visualisation you produce in today's hands-on lab and simulation, document the design decisions you made and the analytical claim the visualisation is intended to support. This log trains the discipline of intentional visualisation design — a critical skill for Central Bank executive reporting.

Chart Title	Chart Type (Matplotlib / Seaborn)	Analytical Claim Being Supported	Data Variables Plotted	Design Decision Made (scale / colour / annotation)	Risk of Misinterpretation and Mitigation

✏ DAY 5 REFLECTION JOURNAL

> Day 5 is the midpoint of the programme. Use this reflection to take stock of your progress across the first five days and set a clear direction for the second half.

In the Phase 1 Simulation, your team produced an integrated analytical report under time pressure. Looking at your Analytical Report Quality Assessment Rubric (Worksheet 5.1), which criterion received the lowest score and what specific technical or process gap caused it?

The Scikit-learn guided practice introduced predictive modelling at a conceptual and operational level. Identify one specific pattern recognition or classification problem in Central Bank data — for example, anomaly detection in transaction data — where a Scikit-learn model could add analytical value, and describe what training data you would need.

Time series decomposition revealed trend, seasonal, and residual components. Which of these three components is most relevant to a financial dataset you currently work with at the Central Bank, and what decision would a clearer understanding of that component improve?

Across the first five days you have built environment configurations, optimised queries, designed stored procedures, built ETL pipelines, and produced statistical visualisations. Which of these five capabilities feels least consolidated in your practice, and what will you do in the second half of the programme to strengthen it?

DAY 6: AUTOMATED REPORTING, WORKFLOW INTEGRATION, AND PHASE 2 SIMULATION

Modules: Module 6: Automated Reporting and Workflow Integration

Day 6 transitions participants into the implementation phase, where they design and build end-to-end automated reporting pipelines that deliver outputs to stakeholders without manual intervention. Task scheduling, email automation, Jinja2 templating, and deployment configuration are combined in the Phase 2 Simulation, where teams construct and stress-test a realistic monthly reporting system aligned to Central Bank operational requirements.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 6: Automated Reporting and Workflow Integration**	Day 6 orientation: connecting Day 5 analytical outputs to automated distribution, introducing the Phase 2 Simulation challenge brief	Facilitated Reflection
09:30 - 10:30	**Module 6: Automated Reporting and Workflow Integration**	Task scheduling using Windows Task Scheduler, cron jobs, and Python scheduling libraries for automated workflow execution	Interactive Lecture
10:30 - 10:45			*Break*

Time	Module	Session / Activity	Method
10:45 - 11:45	**Module 6: Automated Reporting and Workflow Integration**	Email automation, report distribution, and notification systems for stakeholder communication and alert management	Hands-on Lab
11:45 - 13:00	**Module 6: Automated Reporting and Workflow Integration**	Template-based reporting using Jinja2, automated document generation, and professional output formatting	Guided Practice
13:00 - 14:00			*Break*
14:00 - 14:45	**Module 6: Automated Reporting and Workflow Integration**	Configuration management, environment variables, and deployment strategies for production automation systems	Workshop
14:45 - 15:30	**Module 6: Automated Reporting and Workflow Integration**	**★ Exercise: Build automated monthly reporting system that extracts data via T-SQL, processes with Python, and distributes formatted reports**	Simulation & Debrief
15:30 - 15:45			*Break*
15:45 - 16:30			*Group Exercise*
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 6 Learning Outcomes

1. Design a task scheduling architecture using Windows Task Scheduler and Python scheduling libraries to automate recurring workflow execution
2. Implement email automation and notification logic to distribute formatted analytical reports to defined stakeholder groups
3. Develop Jinja2-templated report documents that meet professional formatting standards for Central Bank executive audiences
4. Construct configuration management and environment variable structures that support reliable deployment across development and production systems
5. Execute the automated monthly reporting system exercise by integrating T-SQL extraction, Python processing, and formatted report distribution
6. Evaluate the Phase 2 Simulation pipeline against scheduling reliability, output quality, and stakeholder communication effectiveness benchmarks

Task Scheduling Architecture: Windows Task Scheduler vs Python Libraries

> **SCHEDULING DECISION**
>
> Windows Task Scheduler is appropriate for simple, single-machine workflows where the trigger is time-based and the environment is stable. Python scheduling libraries (APScheduler, schedule) are preferable when the workflow logic itself needs to determine execution timing, handle failures gracefully, or run across multiple environments. For production Central Bank systems, always pair any scheduler with a logging mechanism that records execution start time, completion time, and outcome.

For each scheduling approach discussed, record the setup requirement, failure handling behaviour, and the Central Bank workflow scenario where it is the better choice.

Scheduling Approach	Setup Requirement	Failure Handling Behaviour	Best-Fit Central Bank Scenario	Logging Integration Method

Scheduling Approach	Setup Requirement	Failure Handling Behaviour	Best-Fit Central Bank Scenario	Logging Integration Method

Your Notes:

Jinja2 Report Template Design

> **TEMPLATE STANDARD**
>
> A Jinja2 template for executive reporting should separate structure from data completely. The template file defines layout, headings, and formatting; the Python script injects data at runtime using the render() method. Never hard-code data values in the template. For Central Bank reports, include a template variable for the report generation timestamp and the data extraction date range — these are mandatory for audit trail purposes.

Design the structure of a Jinja2 template for a Central Bank monthly report. For each section, specify the template variable name, the data source (T-SQL query or Python calculation), and the formatting requirement.

Report Section	Jinja2 Variable Name	Data Source (T-SQL / Python)	Formatting Requirement	Mandatory for Audit? Y/N

Your Notes:

WORKSHEET 6.1: Automated Reporting Pipeline Architecture Design

Design the complete architecture of your Phase 2 Simulation automated monthly reporting system. Complete this worksheet before building the system, then annotate it after the simulation run to record what worked, what failed, and what you would change in a production deployment at the Central Bank.

Pipeline Stage	Technology / Method Used	Input	Output	Scheduling Trigger	Failure Handling	Post-Simulation Annotation

Pipeline Stage	Technology / Method Used	Input	Output	Scheduling Trigger	Failure Handling	Post-Simulation Annotation

WORKSHEET 6.2: Phase 2 Simulation Stress-Test and Lessons Learned Log

During the Phase 2 Simulation Debrief, record every failure mode, unexpected behaviour, or quality gap identified when your pipeline was demonstrated or stress-tested by the facilitator. For each issue, document the root cause and the production-hardening measure you would implement. This log directly informs your Day 10 Capstone deployment plan.

Failure Mode / Quality Gap Identified	Pipeline Stage Affected	Root Cause	Impact on Report Quality or Scheduling	Production-Hardening Measure	Priority for Capstone Deployment Plan

✏ DAY 6 REFLECTION JOURNAL

> Day 6 produced a live automated reporting system. Reflect on the gap between what you built in the simulation and what a production-grade Central Bank system would require.

In your Phase 2 Simulation Stress-Test Log (Worksheet 6.2), which failure mode would have the most serious consequence if it occurred in a live Central Bank monthly regulatory report distribution? What single hardening measure would most reduce that risk?

The Jinja2 template you designed separates structure from data. Looking at a report your team currently produces manually at the Central Bank, estimate how many hours per month could be saved by automating it with a similar pipeline — and what would need to change in your team's workflow to make that automation trustworthy?

Configuration management and environment variables were introduced as a deployment discipline. Does your current team have a documented practice for managing credentials and environment-specific settings in scripts? If not, what is the first step you would take to establish one?

The Phase 2 Simulation required your team to demonstrate a live automated report run. What was the most stressful moment during that demonstration, and what does it reveal about the gap between a working prototype and a production-ready system?

DAY 7: API INTEGRATION, EXTERNAL DATA SOURCES, AND DATA QUALITY ASSURANCE

Modules: Module 7: API Integration and External Data Sources

Day 7 extends the Central Bank cohort's data acquisition capability beyond internal databases to the broader data ecosystem. Participants build API consumers, parse JSON and XML payloads, apply web scraping techniques, and implement rigorous data quality validation before loading external data into SQL Server. These skills are critical for integrating market data, regulatory feeds, and third-party financial data sources.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 7: API Integration and External Data Sources**	Day 7 orientation: mapping external data source integration to Central Bank use cases — market data, regulatory APIs, and inter-bank feeds	Facilitated Reflection
09:30 - 10:30	**Module 7: API Integration and External Data Sources**	RESTful API consumption using Python requests library, authentication handling, and rate limiting strategies	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:45	**Module 7: API Integration and External Data Sources**	JSON and XML data processing, parsing techniques, and integration with relational database structures	Hands-on Lab
11:45 - 13:00	**Module 7: API Integration and External Data Sources**	Web scraping fundamentals using BeautifulSoup and Scrapy for automated data collection from web sources	Guided Practice
13:00 - 14:00			*Break*

Time	Module	Session / Activity	Method
14:00 - 15:00	**Module 7: API Integration and External Data Sources**	Data source validation, quality assessment, and integration testing for reliable external data incorporation	Data Lab
15:00 - 15:30	**Module 7: API Integration and External Data Sources**	**★ Exercise: Create integration solution that pulls data from external API, validates quality, and stores in SQL Server database**	Workshop
15:30 - 15:45			*Break*
15:45 - 16:30			*Peer Review*
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 7 Learning Outcomes

1. Implement RESTful API consumption scripts using the Python requests library with authentication handling and rate limiting controls
2. Develop JSON and XML parsing routines that accurately map external data structures to relational database schemas
3. Apply BeautifulSoup web scraping techniques to extract structured data from authorised web sources for database integration
4. Design data source validation and quality assessment procedures that enforce acceptance criteria before database insertion
5. Construct integration testing suites that verify the reliability of external data incorporation workflows end-to-end
6. Execute the API integration exercise to deliver a validated, quality-assured external data load into a SQL Server database

RESTful API Consumption: Authentication and Rate Limiting

> **SECURITY REQUIREMENT**
>
> API credentials must never be hard-coded in Python scripts. Store API keys and tokens in environment variables or a secrets management solution, and load them at runtime using os.environ.get(). For Central Bank integrations with external financial data providers, always confirm the authentication mechanism (API key, OAuth 2.0, or mutual TLS) and the rate limit policy before building the consumer — exceeding rate limits can result in IP blocking or service suspension.

For each external data source relevant to the Central Bank's operations, record the likely authentication mechanism, the rate limit consideration, and the Python requests pattern you would use.

External Data Source Type	Authentication Mechanism	Rate Limit Consideration	Python requests Pattern	Credential Storage Method

Your Notes:

Data Quality Acceptance Criteria Framework

QUALITY GATE PRINCIPLE

External data should never be loaded directly to a production database without passing a defined quality gate. A quality gate specifies: completeness thresholds (minimum percentage of non-null values per field), validity rules (value ranges, format patterns, referential integrity), timeliness criteria (maximum acceptable data age), and consistency checks (cross-field logic validation). Failing any gate criterion should trigger a rejection log entry and an alert — not a silent load.

Define the data quality acceptance criteria for one external data source your team would integrate at the Central Bank. Specify the threshold or rule for each quality dimension.

Quality Dimension	Acceptance Criterion / Threshold	Validation Method (Python / SQL)	Failure Action	Responsible Owner

Your Notes:

WORKSHEET 7.1: API Integration Solution Design and Test Record

Document your API integration solution design and the results of each test run during the workshop exercise and cross-team peer review. This record serves as the technical specification and test evidence for your integration — a format directly applicable to Central Bank change management documentation.

API Endpoint	HTTP Method	Authentication Type	Response Format (JSON/XML)	Parsing Method Used	Quality Gate Applied	Test Result (Pass/Fail)	Rows Loaded to SQL Server

WORKSHEET 7.2: External Data Source Risk and Reliability Assessment

For each external data source your team identified during the Day 7 orientation discussion, assess its reliability and integration risk. Use this assessment to prioritise which sources to integrate first and which require additional due diligence before production use at the Central Bank.

External Data Source	Data Type (Market / Regulatory / Reference)	Availability SLA (if known)	Authentication Complexity	Data Quality Risk (High/Med/Low)	Integration Priority	Due Diligence Required Before Production

✏ DAY 7 REFLECTION JOURNAL

> Day 7 extended your data acquisition capability to external sources. Reflect on the quality and reliability implications of integrating external data into Central Bank systems.

In the cross-team integration testing review, your team ran another team's validation suite against your API solution. What quality gap did their tests expose that your own tests had not caught, and what does that reveal about the blind spots in your current testing approach?

Looking at your External Data Source Risk Assessment (Worksheet 7.2), which source carries the highest data quality risk? What specific quality gate criterion from today's session would you implement first to mitigate that risk before a production integration?

Web scraping with BeautifulSoup was introduced as a data collection technique. Identify one publicly available web source — for example, a regulatory publication or financial statistics page — that the Central Bank currently accesses manually, and describe what a responsible, authorised scraping solution would look like for that source.

The API credential storage principle requires environment variables rather than hard-coded values. Audit your own current scripting practice: do any scripts you have written or inherited contain hard-coded credentials? What is your plan to remediate them?

DAY 8: PERFORMANCE MONITORING, SYSTEM OPTIMIZATION, AND PRODUCTION READINESS

Modules: Module 8: Performance Monitoring and System Optimization

Day 8 equips participants to move beyond building solutions to sustaining and improving them in production. Using DMVs, Query Store, Extended Events, and Python profiling tools, participants establish performance baselines, design monitoring dashboards, and implement logging frameworks that keep enterprise systems healthy and observable. This operational expertise is essential for the Central Bank's expectation of high-availability, auditable database systems.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 8: Performance Monitoring and System Optimization**	Day 8 orientation: framing production monitoring as a strategic obligation for Central Bank data systems and introducing today's observability design challenge	Facilitated Reflection
09:30 - 10:30	**Module 8: Performance Monitoring and System Optimization**	Database performance monitoring using DMVs, Query Store, and Extended Events for proactive system management	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:45	**Module 8: Performance Monitoring and System Optimization**	Python application profiling, memory management, and optimization techniques for efficient script execution	Hands-on Lab
11:45 - 13:00	**Module 8: Performance Monitoring and System Optimization**	Logging frameworks, error tracking, and debugging strategies for maintainable production systems	Guided Practice
13:00 - 14:00			*Break*
14:00 - 15:00	**Module 8: Performance Monitoring and System Optimization**	Capacity planning, scalability assessment, and performance baseline establishment for growing data volumes	Case Study Analysis
15:00 - 15:30	**Module 8: Performance Monitoring and System Optimization**	**★ Exercise: Design comprehensive monitoring dashboard that tracks both database performance and Python workflow execution metrics**	Data Lab
15:30 - 15:45			*Break*
15:45 - 16:30			*Group Exercise*
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 8 Learning Outcomes

1. Demonstrate the use of DMVs, Query Store, and Extended Events to proactively identify and diagnose database performance issues
2. Implement Python application profiling and memory management techniques to optimise script execution efficiency in production workflows
3. Construct logging frameworks and error tracking systems that support maintainable, debuggable production database applications

4. Formulate capacity planning and scalability assessments using established performance baselines and projected data volume growth
5. Design a comprehensive monitoring dashboard that tracks database performance metrics and Python workflow execution status simultaneously
6. Assess the monitoring dashboard produced in the lab exercise against defined alerting thresholds and operational observability standards

DMVs, Query Store, and Extended Events: Monitoring Tool Selection

> **MONITORING HIERARCHY**
>
> Dynamic Management Views (DMVs) provide point-in-time snapshots of current server state — useful for immediate diagnosis. Query Store persists query performance history across server restarts — essential for trend analysis and regression detection after deployments. Extended Events is the low-overhead event tracing framework for capturing specific, high-frequency events without the performance cost of SQL Profiler. For Central Bank production systems, all three should be active and their outputs reviewed on a defined schedule.

For each monitoring tool, record the key DMV, Query Store view, or Extended Event session you would configure for a Central Bank production database, and the specific performance question it answers.

Monitoring Tool	Key View / Session Name	Performance Question It Answers	Recommended Review Frequency	Alert Threshold to Configure

Your Notes:

Python Logging Framework Design

> **LOGGING STANDARD**
>
> Python's built-in logging module supports five severity levels: DEBUG, INFO, WARNING, ERROR, and CRITICAL. For production ETL and automation scripts at the Central Bank, configure at minimum: INFO-level logging for every successful pipeline stage completion, WARNING for data quality anomalies that do not halt execution, ERROR for exceptions that are caught and handled, and CRITICAL for failures that halt the pipeline. Write logs to both a rotating file handler and a database logging table for audit trail purposes.

Design the logging configuration for one of the Python scripts you built earlier in the programme. Specify the log level, message format, handler type, and retention policy for each logging event.

Logging Event	Severity Level	Log Message Format	Handler Type (File / DB / Console)	Retention Policy

Logging Event	Severity Level	Log Message Format	Handler Type (File / DB / Console)	Retention Policy

Your Notes:

WORKSHEET 8.1: Performance Baseline Establishment Record

During the capacity planning case study and monitoring dashboard lab, establish a performance baseline for the database and Python workflow environment you are working with. Record each metric, its current measured value, the acceptable operating range, and the alert threshold you would configure. This baseline record is a reusable template for establishing production baselines at the Central Bank.

Performance Metric	Measurement Tool (DMV / Query Store / Python profiler)	Current Measured Value	Acceptable Operating Range	Alert Threshold	Escalation Action When Threshold Breached

WORKSHEET 8.2: Monitoring Dashboard Architecture Design

Design the architecture of the comprehensive monitoring dashboard you build in the Data Lab exercise. For each dashboard panel, specify what it displays, the data source, the refresh frequency, and the alerting logic. This design document is a deliverable you can present to your manager as a production monitoring proposal for Central Bank database systems.

Dashboard Panel Name	Metric Displayed	Data Source (DMV / Query Store / Python log)	Refresh Frequency	Alert Condition	Notification Method (Email / Log / Alert)

Dashboard Panel Name	Metric Displayed	Data Source (DMV / Query Store / Python log)	Refresh Frequency	Alert Condition	Notification Method (Email / Log / Alert)

✏ DAY 8 REFLECTION JOURNAL

> Day 8 shifted focus from building to sustaining. Reflect on the gap between your current monitoring practices and the production-grade observability standards introduced today.

In your Performance Baseline Establishment Record (Worksheet 8.1), which metric had the largest gap between its current measured value and the acceptable operating range? What does that gap tell you about the current health of the database environment you work with at the Central Bank?

The Python logging framework design exercise required you to specify a log level and handler for each event type. Looking at the ETL script you built on Day 4, how many of those logging events are currently implemented in your script? What is the most critical missing log entry?

During the dashboard design review, the facilitator stress-tested your alerting logic. Which alert condition failed the stress test, and what change to the threshold or escalation action would make it production-ready?

Capacity planning requires projecting data volume growth. Based on your knowledge of Central Bank data volumes, estimate the growth rate of one key dataset over the next 24 months and describe what infrastructure or query optimisation change that growth would require.

DAY 9: SECURITY, COMPLIANCE, BEST PRACTICES, AND ENTERPRISE DEPLOYMENT PLANNING

Modules: Module 9: Security, Compliance, and Best Practices | Module 10: Enterprise Deployment and Strategic Planning

Day 9 addresses the non-negotiable dimensions of enterprise database programming — security, regulatory compliance, and disciplined deployment. Participants conduct a security assessment of real procedures and scripts, implement recommended hardening measures, and then pivot to deployment planning, change management, and ROI definition. For the Central Bank of Lesotho, operating in a regulated financial environment, these skills are as critical as any technical capability.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 9: Security, Compliance, and Best Practices**	Day 9 orientation: security and compliance as strategic imperatives in a central banking environment — framing today's assessment and deployment planning work	Facilitated Reflection
09:30 - 10:30	**Module 9: Security, Compliance, and Best Practices**	Database security principles, role-based access control, and SQL injection prevention in stored procedures and dynamic queries	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:15	**Module 9: Security, Compliance, and Best Practices**	Python security considerations, credential management, and secure coding practices for database applications	Guided Practice
11:15 - 12:00	**Module 9: Security, Compliance, and Best Practices**	Data privacy regulations, audit trail implementation, and compliance documentation for regulated industries	Case Study Analysis
12:00 - 13:00	**Module 9: Security, Compliance, and Best Practices**	**★ Exercise: Conduct security assessment of existing database procedures and Python scripts, implementing recommended security enhancements**	Hands-on Lab
13:00 - 14:00			*Break*
14:00 - 14:45	**Module 9: Security, Compliance, and Best Practices**	Code review processes, testing strategies, and quality assurance procedures for reliable database programming solutions	Workshop
14:45 - 15:30	**Module 10: Enterprise Deployment and Strategic Planning**	Deployment planning, environment management, and release procedures for database and Python application changes	Group Exercise
15:30 - 15:45			*Break*
15:45 - 16:30	**Module 10: Enterprise Deployment and Strategic Planning**	Change management strategies, user training programs, and adoption planning for new automated systems	Case Study Analysis
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 9 Learning Outcomes

1. Implement role-based access control and SQL injection prevention measures across stored procedures and dynamic queries to meet financial sector security standards
2. Assess Python credential management and secure coding practices against a defined threat model for database-connected applications
3. Formulate audit trail and compliance documentation structures that satisfy data privacy regulations applicable to Central Bank operations

4. Design a code review and quality assurance process that enforces security and reliability standards across the development lifecycle
5. Construct a deployment plan and change management strategy for rolling out database and Python automation changes across production environments
6. Evaluate performance KPIs, success metrics, and ROI calculation frameworks to demonstrate measurable return on database programming investments

Role-Based Access Control and SQL Injection Prevention

SECURITY PRINCIPLE

In SQL Server, the principle of least privilege requires that every database role is granted only the permissions required to perform its defined function — nothing more. Application service accounts should never have db_owner or sysadmin rights. For dynamic SQL, always use sp_executesql with parameterised inputs rather than string concatenation — this is the single most effective SQL injection prevention measure available in T-SQL.

Map the database roles required for a Central Bank database programming environment. For each role, specify the minimum permissions required, the objects those permissions apply to, and the SQL injection risk if the role is over-privileged.

Database Role	Minimum Permissions Required	Objects Permissions Apply To	Injection Risk if Over-Privileged	Review Frequency

Your Notes:

Deployment Planning and Release Procedure Framework

DEPLOYMENT DISCIPLINE

Every database or Python application change deployed to a Central Bank production environment must pass through four gates: (1) Development — unit tested and peer reviewed; (2) Test — integration tested against a production-equivalent dataset; (3) Staging — performance tested and security scanned; (4) Production — deployed via a documented release procedure with a rollback plan. Skipping any gate increases the risk of a production incident that affects regulatory reporting or financial operations.

Design the deployment gate checklist for a stored procedure change at the Central Bank. For each gate, specify the required checks, the approver, and the rollback trigger condition.

Deployment Gate	Required Checks	Approver / Sign-Off Role	Rollback Trigger Condition	Documentation Required

Deployment Gate	Required Checks	Approver / Sign-Off Role	Rollback Trigger Condition	Documentation Required

Your Notes:

WORKSHEET 9.1: Security Assessment Findings and Remediation Plan

During the security assessment hands-on lab, evaluate each stored procedure and Python script against the security criteria below. Record every finding, its severity, and the specific remediation action you implemented or recommended. This assessment report is a deliverable you can present to your information security team at the Central Bank.

Object Assessed (SP / Script Name)	Security Criterion Evaluated	Finding (Pass / Fail / Partial)	Severity (Critical / High / Medium / Low)	Remediation Action Implemented	Residual Risk After Remediation

WORKSHEET 9.2: Change Management and Adoption Planning Template

Using the change management case study and deployment planning group exercise as your reference, design the change management plan for rolling out one of the automated systems you built during this programme to the broader Central Bank team. Complete all columns — this template is directly reusable for your Day 10 Capstone deployment plan.

Change Component	Affected Stakeholder Group	Communication Action Required	Training Required (Y/N / Description)	Adoption Risk	Mitigation Strategy	Success Indicator

Change Component	Affected Stakeholder Group	Communication Action Required	Training Required (Y/N / Description)	Adoption Risk	Mitigation Strategy	Success Indicator

✏ DAY 9 REFLECTION JOURNAL

> Day 9 confronted the security and compliance obligations that govern all database programming work in a regulated financial institution. Reflect honestly on the gap between current practice and the standards introduced today.

In your Security Assessment Findings (Worksheet 9.1), how many Critical or High severity findings did you identify in the procedures and scripts you assessed? What does the pattern of those findings tell you about the most common security gap in your team's current coding practices?

The deployment gate framework requires four sequential checks before any change reaches production. Describe the current deployment process for a database change at the Central Bank — how many of the four gates does it currently include, and which gate is most frequently skipped or compressed under time pressure?

The compliance documentation session covered audit trail requirements for regulated industries. Looking at the trigger-based audit log you designed on Day 3, does it capture all the fields required to satisfy a regulatory audit of a financial data change? What is missing?

The change management case study presented a scenario where a new automated system was technically sound but poorly adopted by end users. Identify one automated system you plan to deploy at the Central Bank after this programme and describe the single most important adoption risk you need to manage.

DAY 10: STRATEGIC ROADMAPS, CAPSTONE PRESENTATIONS, AND PROGRAMME CLOSE

Modules: Module 10: Enterprise Deployment and Strategic Planning

Day 10 brings the full programme to its synthesis conclusion, where participants consolidate ten days of technical and strategic learning into enterprise-ready deployment plans and roadmaps. Capstone presentations challenge each participant to present a comprehensive database programming initiative — complete with KPIs, ROI projections, and a technology roadmap — to a panel audience simulating Central Bank leadership. The day closes with certificate recognition and a commitment to continued professional growth.

Daily Schedule

Time	Module	Session / Activity	Method
09:00 - 09:30	**Module 10: Enterprise Deployment and Strategic Planning**	Day 10 orientation: final programme review, capstone presentation briefing, and panel assessment criteria walkthrough	Administrative
09:30 - 10:30	**Module 10: Enterprise Deployment and Strategic Planning**	Performance KPI definition, success metrics establishment, and ROI calculation for database programming investments	Interactive Lecture
10:30 - 10:45			*Break*
10:45 - 11:45	**Module 10: Enterprise Deployment and Strategic Planning**	Strategic roadmap development, technology evaluation, and future capability planning for evolving data requirements	Design Sprint
11:45 - 13:00	**Module 10: Enterprise Deployment and Strategic Planning**	Exercise: Create comprehensive deployment plan and success metrics framework for organization-wide database programming initiative	Workshop
13:00 - 14:00			*Break*
14:00 - 15:30	**Module 10: Enterprise Deployment and Strategic Planning**	**★ Capstone Presentations Round 1 & 2: participants present comprehensive deployment plans and strategic roadmaps to the simulated executive panel; panel poses challenge questions on security, KPIs, and ROI assumptions**	Simulation & Debrief
15:30 - 15:45			*Break*
15:45 - 16:30	**Module 10: Enterprise Deployment and Strategic Planning**	Capstone Presentations & Action Planning	Assessment
16:30 - 17:00		Closing Ceremony & Certificates	Administrative

Day 10 Learning Outcomes

1. Formulate performance KPI definitions and success metrics frameworks that quantify the ROI of database programming investments for Central Bank leadership
2. Create a strategic technology roadmap that evaluates emerging tools and maps future capability requirements to evolving organisational data needs
3. Synthesise T-SQL, Python, security, automation, and monitoring knowledge into a coherent enterprise-grade database programming initiative proposal
4. Present a comprehensive deployment plan to a simulated executive panel, demonstrating command of both technical detail and strategic business impact
5. Assess peer capstone proposals against defined criteria — technical soundness, security compliance, measurability, and stakeholder alignment
6. Demonstrate professional readiness to implement an organisation-wide database programming initiative by defending design decisions under panel questioning

KPI Definition and ROI Calculation Framework

ROI CALCULATION PRINCIPLE

A credible ROI calculation for a database programming investment requires three inputs: (1) the cost of the investment (staff time, licences, infrastructure, training); (2) the quantified benefit (hours saved per month × staff cost rate, error reduction rate × cost per error, reporting cycle time reduction); and (3) the payback period (investment cost ÷ monthly benefit). For Central Bank leadership, always present ROI with a conservative, a base-case, and an optimistic scenario — and state the assumptions behind each.

Define the KPIs and ROI inputs for your capstone initiative. Complete all cells — vague KPIs will not survive panel questioning.

KPI Name	Measurement Method	Baseline Value (Current State)	Target Value (12 Months)	Data Source for Measurement	ROI Contribution (Quantified)

Your Notes:

Strategic Technology Roadmap Structure

ROADMAP DISCIPLINE

A technology roadmap for database programming capability at the Central Bank should cover three horizons: Horizon 1 (0-6 months) — stabilise and optimise existing systems using skills from this programme; Horizon 2 (6-18 months) — extend automation and integration capability to additional data domains; Horizon 3 (18-36 months) — evaluate emerging technologies (cloud-native databases, advanced analytics platforms) against evolving regulatory and operational requirements. Each horizon must have named owners, budget estimates, and measurable milestones.

Draft your three-horizon technology roadmap for the Central Bank database programming initiative. For each horizon, specify the capability goal, the key technology or practice to be adopted, and the measurable milestone that confirms the horizon is complete.

Horizon	Timeframe	Capability Goal	Key Technology / Practice	Measurable Milestone	Named Owner

Your Notes:

WORKSHEET 10.1: Capstone Peer Assessment Scorecard

During Capstone Presentations Round 1 and 2, use this scorecard to assess one peer's presentation. Score each criterion from 1 (does not meet standard) to 4 (exceeds standard). Record specific evidence for each score and one constructive improvement recommendation. Return the completed scorecard to the presenter at the end of the session.

Assessment Criterion	Score (1-4)	Specific Evidence Observed	Constructive Improvement Recommendation

✏ DAY 10 REFLECTION JOURNAL

> This is your final reflection in the programme. Write with the same honesty and specificity you have brought to every previous journal entry — and write as if your manager will read this page next week.

In your capstone presentation, the panel challenged one of your KPI definitions or ROI assumptions. Was the challenge valid? Revise the challenged KPI or assumption in writing here, incorporating the panel's feedback.

Looking back at your Day 1 Environment Readiness Assessment (Worksheet 1.1) and your Day 10 KPI definition grid, describe the most significant technical capability you have developed over the ten days — and name the specific exercise or worksheet that produced that development.

Your 30-60-90 Day Action Plan commits you to specific actions in your first three months back at the Central Bank. Which action in the first 30 days is most likely to face organisational resistance, and what is your specific strategy for managing that resistance?

The programme covered T-SQL, Python, automation, statistical analysis, API integration, performance monitoring, security, and strategic deployment. Which of these eight capability areas do you most need to continue developing independently after the programme, and what is the first concrete step you will take in that direction within the next two weeks?

CAPSTONE PROJECT

CAPSTONE TEMPLATE: Enterprise Database Programming Initiative Proposal — Central Bank of Lesotho

All eight sections of this capstone template are required. Your completed capstone must integrate evidence and outputs from prior days' worksheets — specific references are noted in each section. This document is designed to be presented to a simulated Central Bank executive panel on Day 10 and submitted to your line manager within five working days of returning to the office. Vague or unsupported claims will not survive panel questioning.

Section 1: Initiative Overview and Problem Statement (draws on Day 1 environment assessment and Day 2 query optimisation benchmark log)

Describe the specific database programming problem or opportunity your initiative addresses at the Central Bank. Reference the environment readiness gaps identified in Worksheet 1.1 and the performance bottlenecks documented in Worksheet 2.1. State the current-state pain point in measurable terms — avoid vague language such as 'improve efficiency'.

Section 2: Technical Solution Architecture (draws on Days 3, 4, and 6 worksheets)

Map each technical component of your proposed solution — stored procedures, ETL scripts, automated reporting pipeline, API integrations — to the technology used, the day it was built or designed, and the security control applied. Reference Worksheets 3.1, 4.1, and 6.1.

Solution Component	Technology / Method	Day Built / Referenced	Integration Point with Other Components	Security Control Applied

Solution Component	Technology / Method	Day Built / Referenced	Integration Point with Other Components	Security Control Applied

Section 3: Security and Compliance Framework (draws on Day 9 security assessment)

Document the security and compliance measures built into your initiative. Reference the findings and remediations from Worksheet 9.1. For each requirement, specify how compliance will be verified and which regulatory or internal standard it satisfies.

Security / Compliance Requirement	Implementation Measure	Verification Method	Responsible Role	Regulatory Reference

Section 4: Performance Monitoring and Observability Plan (draws on Day 8 monitoring dashboard design)

Define the monitoring and observability plan for your initiative in production. Reference the performance baseline record (Worksheet 8.1) and monitoring dashboard architecture (Worksheet 8.2). Every metric must have a defined baseline and alert threshold.

Metric Monitored	Monitoring Tool	Baseline Value	Alert Threshold	Escalation Procedure

Section 5: Deployment and Change Management Plan (draws on Days 9 and 6 worksheets)

Outline the phased deployment plan for your initiative, incorporating the deployment gate framework from Day 9 and the change management template from Worksheet 9.2. Each phase must have a named go/no-go criterion before proceeding to the next.

Deployment Phase	Actions Required	Stakeholder Group Affected	Change Management Activity	Go / No-Go Criterion

Deployment Phase	Actions Required	Stakeholder Group Affected	Change Management Activity	Go / No-Go Criterion

Section 6: KPI Framework and ROI Projection (draws on Day 10 KPI definition grid)

Present the KPI framework and ROI projection for your initiative. Reference the KPI definition grid completed in today's session notes. All ROI estimates must state their assumptions explicitly — unsupported figures will be challenged by the panel.

KPI Name	Baseline Value	12-Month Target	Measurement Method	Conservative ROI Estimate	Base-Case ROI Estimate

Section 7: Strategic Technology Roadmap (draws on Day 10 three-horizon roadmap)

Present the three-horizon technology roadmap for your initiative. Reference the roadmap drafted in today's session notes. Each horizon must have a named owner and a measurable milestone — not a vague aspiration.

Horizon	Timeframe	Capability Goal	Key Milestone	Budget Estimate (indicative)	Named Owner

Section 8: Executive Summary and Call to Action

Write a concise executive summary of your initiative — no more than eight lines — that a Central Bank executive could read in 90 seconds and understand: what the initiative does, what it costs, what it delivers, and what decision is required from leadership. End with a specific, dated call to action: what you are asking the panel to approve, fund, or endorse, and by when.

30-60-90 DAY ACTION PLAN

Transfer your training into workplace impact. Complete each section with specific, measurable actions.

First 30 Days: Quick Wins — Stabilise and Apply

Action	Specific Steps	Resources Needed	Deadline	Accountability Partner

Days 31-60: Deeper Implementation — Automate and Integrate

Action	Specific Steps	Resources Needed	Deadline	Success Indicator

Days 61-90: Systemic Change — Embed and Scale

Action	Specific Steps	Resources Needed	Deadline	Organisational Impact

Action	Specific Steps	Resources Needed	Deadline	Organisational Impact

My Accountability Partner

Name:

Organization:

Email:

Phone:

Check-in Schedule:

REFERENCE SECTION

Key Frameworks & Standards

• ISO/IEC 27001:2022 — Information Security Management Systems: Specifies requirements for establishing, implementing, maintaining, and continually improving an information security management system — directly applicable to database access control, credential management, and audit trail requirements covered in Module 9.

• NIST SP 800-53 Rev 5 — Security and Privacy Controls for Information Systems and Organizations: Provides a catalogue of security and privacy controls including database access management, audit and accountability, and system and communications protection — relevant to the Central Bank's regulatory compliance obligations.

• NIST Risk Management Framework (RMF) Rev 2 — A structured process for integrating security and risk management activities into the system development lifecycle — applicable to the deployment planning and change management work in Module 10.

• ISO/IEC 25010:2011 — Systems and Software Quality Requirements and Evaluation (SQuaRE): Defines software product quality characteristics including performance efficiency, reliability, and maintainability — provides the quality vocabulary for the code review and QA procedures in Module 9.

• DAMA-DMBOK2 — Data Management Body of Knowledge, Second Edition: The authoritative reference framework for data management disciplines including data quality, data integration, data security, and metadata management — underpins the ETL, data quality, and governance concepts across Modules 4, 7, and 9.

• COBIT 2019 — Control Objectives for Information and Related Technologies: A governance and management framework for enterprise IT that addresses performance measurement, risk management, and IT investment ROI — directly supports the KPI definition and ROI calculation work in Module 10.

• Microsoft SQL Server Documentation — Query Store Best Practices (docs.microsoft.com): Microsoft's official guidance on configuring and using Query Store for performance monitoring, regression detection, and query plan management — the authoritative reference for the DMV and Query Store content in Module 8.

• PEP 8 — Style Guide for Python Code (python.org/dev/peps/pep-0008): The official Python style guide defining coding conventions for readability and maintainability — the baseline standard for the Python secure coding and code review practices in Modules 9 and 3.

• OWASP SQL Injection Prevention Cheat Sheet — Open Web Application Security Project guidance on preventing SQL injection through parameterised queries, stored procedures, and input validation — the primary reference for the SQL injection prevention content in Modules 3 and 9.

• Basel Committee on Banking Supervision — Principles for Effective Risk Data Aggregation and Risk Reporting (BCBS 239): Defines principles for data accuracy, completeness, timeliness, and adaptability in banking risk reporting — provides the regulatory context for data quality and reporting automation requirements at the Central Bank of Lesotho.

Essential Tools Quick Reference

Tool	Purpose	Access / URL
SQL Server Management Studio (SSMS)	Primary IDE for T-SQL development, query execution plan analysis, index management, and database administration — used throughout Modules 1, 2, 3, and 8	https://learn.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms
Visual Studio Code with Python Extension	Lightweight, cross-platform Python IDE supporting IntelliSense, debugging, and Git integration — used for Python development across Modules 1, 4, 5, 6, and 7	https://code.visualstudio.com
pandas (Python library)	Core Python library for DataFrame-based data manipulation, cleaning, and transformation — central to ETL workflows in Module 4 and analytical reporting in Module 5	https://pandas.pydata.org
SQLAlchemy	Python SQL toolkit and ORM providing database-agnostic connectivity, connection pooling, and pandas integration — used for production-grade database connectivity in Module 4	https://www.sqlalchemy.org
Matplotlib	Foundational Python plotting library for creating static, animated, and interactive visualisations — used for analytical dashboards and report charts in Module 5	https://matplotlib.org
Seaborn	Statistical data visualisation library built on Matplotlib, providing high-level chart types for distribution, correlation, and categorical analysis — used in Module 5	https://seaborn.pydata.org
Scikit-learn	Python machine learning library providing classification, regression, clustering, and model evaluation tools at conceptual and operational depth — used in Module 5	https://scikit-learn.org
Jinja2	Python templating engine for generating dynamic HTML, text, and document outputs — used for automated report generation in Module 6	https://jinja.palletsprojects.com
Git (version control)	Distributed version control system for tracking changes to SQL scripts and Python code, supporting code review and audit trail requirements — introduced in Module 1	https://git-scm.com
Python requests library	Standard Python HTTP library for consuming RESTful APIs, handling authentication, and managing rate limiting — used for external data integration in Module 7	https://requests.readthedocs.io

Glossary of Key Terms

Common Table Expression (CTE): A named temporary result set defined within a single T-SQL statement using the WITH clause. CTEs improve query readability and support recursive queries but do not inherently cache results — the optimizer may re-evaluate the CTE multiple times within the same query.

Dynamic Management View (DMV): A system view in SQL Server that returns server state information about the health, performance, and configuration of a running SQL Server instance. DMVs are prefixed with sys.dm_ and provide point-in-time diagnostic data without requiring trace or profiler overhead.

ETL (Extract, Transform, Load): A data integration pattern in which data is extracted from one or more source systems, transformed to meet target schema and quality requirements, and loaded into a destination database or data warehouse. In this programme, ETL pipelines are built using T-SQL for extraction and Python (pandas, SQLAlchemy) for transformation and load.

Execution Plan: A graphical or XML representation of the sequence of operations SQL Server performs to execute a query. Execution plans show operators (scans, seeks, joins, sorts), estimated and actual row counts, and cost percentages — the primary diagnostic tool for query performance tuning.

Parameterised Query: A query in which user-supplied values are passed as typed parameters rather than concatenated into the SQL string. Parameterised queries (using sp_executesql in T-SQL or prepared statements in Python) are the primary defence against SQL injection attacks.

Query Store: A SQL Server feature that automatically captures query text, execution plans, and runtime statistics, persisting them in the user database. Query Store enables performance regression detection after index changes or upgrades and supports plan forcing to stabilise query performance.

Role-Based Access Control (RBAC): A security model in which database permissions are assigned to named roles rather than individual users. Users are then granted membership in the appropriate role. RBAC simplifies permission management, supports the principle of least privilege, and is the standard access control model for enterprise SQL Server environments.

Stored Procedure: A precompiled, named collection of T-SQL statements stored in the database and executed as a unit. Stored procedures support parameter handling, return values, error handling, and execution plan caching — making them the preferred mechanism for encapsulating business logic in enterprise database applications.

Window Function: A T-SQL function that computes a value for each row based on a defined window (partition) of related rows, without collapsing the result set. Window functions (ROW_NUMBER, RANK, SUM OVER, LAG, LEAD) are essential for running totals, period-over-period comparisons, and ranking in financial reporting queries.

DataFrame (pandas): The primary two-dimensional, labelled data structure in the pandas Python library, analogous to a database table or spreadsheet. DataFrames support vectorised operations, missing value handling, grouping, merging, and reshaping — making them the standard structure for Python-based data transformation and analysis.

Jinja2 Template: A text-based template file processed by the Jinja2 Python library, in which placeholder variables ({{ variable_name }}) and control structures ({% for %}, {% if %}) are replaced with runtime data to generate dynamic output documents such as HTML reports or formatted text files.

Least Privilege Principle: A security design principle requiring that every user, process, or system component is granted only the minimum permissions necessary to perform its defined function. In database programming, this means application service accounts should have execute-only rights on stored procedures, not direct table-level read/write permissions.

PARTICIPANT NOTES

Use the space below to capture key insights, action items, questions, and personal reflections throughout your training.

Session Notes

Record key takeaways, insights, and ideas from each session.

Action Items & Follow-ups

Track tasks, commitments, and follow-up actions you identify during the training.

Action Item	Priority	Deadline	Status

Questions & Parking Lot

Capture questions that arise during sessions for later discussion or self-research after the program.

Question	Session/Context	Answer / Resolution

Networking & Contacts

Record contact details and key discussion points from fellow participants, facilitators, and guest speakers.

Name	Organization / Role	Email / Phone	Discussion Topic

General Notes

Open space for diagrams, mind maps, sketches, and any additional notes.

"Where in your current role do you accept data quality, security, or performance risk implicitly — and who else in the Central Bank would know if you did?"